POS MOBILE · CARTIVO

POS Mobile — Technical Manual

Architecture, environment, local database, build and deployment for the Flutter POS app.

Written for

Developers, Integrators & IT Staff

App v1.2.0 · Manual rev. 1 · September 2026

Confidential — for authorized personnel

Contents

Use your PDF reader's bookmarks / outline panel to jump between sections.

> **NOTE** The app lives in the `mobile/` folder of the repository. A Flutter SDK (Dart 3.7+), Android tooling, and a populated `.env` file are assumed.

1. Overview & tech stack

POS Mobile is an **offline-first Flutter application** in the `mobile/` folder of the POS monorepo. It targets Android primarily (also builds for Windows). All business data lives in a local SQLite database; the only network dependency is the live online-orders feed and its device registration.

Concern	Choice
Language / SDK	Dart `^3.7.2`, Flutter
Local database	`drift` + `drift_flutter` (SQLite), schema version **14**
State management	`hooks_riverpod` + `flutter_hooks`
Navigation	`go_router` (hand-written routes, no codegen router)
Models / JSON	`dart_mappable` (`*.mapper.dart`)
Config	`envied` — `.env` baked in at build time (`env.g.dart`)
Auth	local PIN, `bcrypt` hashes; no server, no session persistence
Realtime	`web_socket_channel`, `connectivity_plus`, `flutter_local_notifications`
HTTP	`dio` (device registration, order status updates)
Printing	`esc_pos_utils_plus`, `print_bluetooth_thermal`, a platform channel for a built-in printer, `pdf` / `printing`
Backups	`archive` (zip), `media_store_plus` (Downloads), `workmanager` (periodic job)
Email	`mailer` (SMTP via Gmail app password)
Charts	`fl_chart` (reports screen, currently hidden)

> **NOTE** App identity: package `com.dpo.mobile`, Android label **POS MOBILE**, in-app brand **CARTIVO**, `MaterialApp` title `POS Mobile`. pubspec version `1.2.0+3`.

2. Repository layout

```
mobile/
  lib/
    main.dart                 app bootstrap (DB, seeder, backup, notifications)
    config/environment/       AppEnv interface + Env (envied) + env.g.dart
    core/
      database/               Drift: app_database.dart, tables/, daos/
      navigation/router.dart  GoRouter + redirect guard
      seeder/                 AdminSeeder (default admin / PIN 000000)
      services/               backup/, notifications/, printing, bluetooth, csv email
      workers/                backup_worker.dart (WorkManager dispatcher)
      csv/                    CSV importers + report exporter
      theme/  widgets/  utils/  providers/  result/  crypto/
    data/
      backend_api/            sources/ schemas/ errors/ (online-orders API)
      secure_storage/         flutter_secure_storage wrappers
    features/<name>/          one folder per feature (see below)
  android/  windows/  assets/
  .env / .env.sample
```

Feature module layout

```
features/<feature>/
  entities/       immutable domain models (+ .mapper.dart)
  repositories/   abstract interface + Impl (calls DAO / API source)
  state/          Riverpod notifiers / providers
  use_cases/      (optional) single-purpose operations
  view/           screens + dialogs (HookConsumerWidget)
```

Features present: `auth`, `dashboard`, `ordering`, `transactions`, `orders` + `live_orders`, `cashier_accounting` (x_reading / z_reading / daily_report), `inventory`, `users`, `settings`, `reports` (built but not linked from the dashboard).

> **NOTE** Clean-architecture rule: `lib/data/` must never import `lib/features/`. Error translation (`DioException` → `ApiException`) lives in the data layer.

3. Environment configuration

Config is compiled in via `envied`. Copy `.env.sample` to `.env` and fill it before building — a missing `.env` fails code generation.

Variable	Purpose	Obfuscated
`ORDERS_LIVE_FEED_WS_URL`	WebSocket base URL for the live orders feed.	no
`ORDERS_EVENTS_API_BASE_URL`	REST base URL for order-status updates & auth token exchange (often the same host).	no
`CLIENT_ID`	Client identifier presented to the orders service.	no
`WEBHOOK_SECRET`	Shared secret for the orders `/auth/token` exchange.	**yes**
`CSV_EXPORT_PASSWORD`	If non-empty, users must enter it before downloading any report CSV. Empty = no gate.	**yes**
`SENDER_EMAIL`	Gmail address that sends report CSV emails.	no
`SENDER_APP_PASSWORD`	Gmail **App Password** (needs 2-Step Verification on the sender account) — not the normal password.	**yes**

> **NOTE** Obfuscated fields are XOR-masked in `env.g.dart`, not encrypted — they deter casual inspection of the APK, nothing more. Rotate them like any embedded secret.

New variables must be added in three places: `.env.sample`, the `AppEnv` interface, and as an `@EnviedField()` on `Env` in `lib/config/environment/`. Then re-run `build_runner`.

`Env()` is provided at the root via `appEnvProvider.overrideWithValue(Env())` in `main.dart`.

4. Development setup

```
cd mobile
cp .env.sample .env          # then fill in the values
flutter pub get
dart run build_runner build --delete-conflicting-outputs

flutter run -d android        # or: flutter run -d windows
dart analyze                  # static analysis (flutter_lints)
```

> **NOTE** There is a `dependency_overrides` pin on `path_provider_android: 2.2.22` — keep it unless you verify a newer version works.

Code generation

Re-run `build_runner` after touching any of:

Generator	Produces	Trigger
`drift_dev`	`*.g.dart` for the database	table / DAO changes
`dart_mappable_builder`	`*.mapper.dart`	any `@MappableClass` model
`envied_generator`	`env.g.dart`	`.env` or `Env` field changes

Never hand-edit `*.g.dart` or `*.mapper.dart`.

5. Local database

`AppDatabase` (`lib/core/database/app_database.dart`) opens a Drift/SQLite database named `mobile_pos` via `driftDatabase(name: 'mobile_pos')`. `schemaVersion = 14`.

- **Migrations** — `MigrationStrategy` with an `onUpgrade` that steps `from → to`. Add a new block per version bump; never `synchronize`-style auto-migrate.
- **Seeding** — `AdminSeeder(db).seed()` runs in `main()`. If no admin row exists it inserts one (`name: 'Admin'`, `role: 'admin'`, PIN `000000` bcrypt-hashed, `isPinChanged: false`). The migration also seeds a default `Regular` product variant path.
- **Product images** — stored as files under the app documents dir (`product_images/`); the DB holds the path or a remote URL in `products.image_url`.

Table reference (key columns)

Table	Purpose / notable columns
`users`	`name, role, pinHash, isActive, employeeId, phone, avatarUrl, isPinChanged`
`store_info`	`storeId, storeName, address, taxRate, currency (PHP), receiptFooter, tin, terminalName`
`payment_methods`	`label, accountName, accountNumber, sortOrder`
`product_groups`	categories: `name, sortOrder, isActive`
`products`	`groupId, name, isAvailable, imageUrl, sortOrder`
`product_variants`	`productId, name, price, isDefault, isActive`
`modifier_groups`	`name, isRequired, maxSelections, isActive`
`modifier_options`	`groupId, name, additionalPrice, isActive`
`product_modifier_groups`	join table product ↔ modifier group (unique pair)
`sales`	`cashierId, total, discount, status, type, createdAt, soNumber, voidReason, voidedAt`
`sale_items`	`saleId, productId, variantName, qty, unitPrice, discountType, discountBeneficiaryId/Name, discountAmount, vatExemptAmount`
`sale_item_modifiers`	`itemId, modifierName, additionalPrice`
`payments`	`saleId, method, amount, cashReceived, reference, createdAt`
`refunds` / `refund_items`	refund records (UI currently exposes void, not partial refund)
`x_readings`	per-cashier snapshot: totals, breakdown JSON blobs, VAT, cash
`z_readings`	store close: `zCounter`, begin/end balance, closedBy / authorizedBy, salesByCashier JSON
`daily_reports`	per-cashier VAT / cash summary + `salesByProductJson`, `cashLedgerJson`
`order_events`	one row per online order (`orderId` PK), latest `eventType` + `payload` JSON. Not the local sales ledger.

6. Authentication & roles

- PINs are 6 digits, hashed with `bcrypt` in `users.pinHash`. No plaintext, no server.
- **No session persistence.** `AuthNotifier.build()` returns `AuthInitial` every launch — the app always starts at `/login`.
- `isPinChanged == false` → `AuthAuthenticated.mustChangePin` → the router forces `/setup-pin` and won't release to `/dashboard` until a new PIN (≠ `000000`) is set and confirmed.
- `verifyPinForUser(userId, pin)` backs the in-place supervisor authorization for voids and Z-Reading closes (no logout).
- `isAdminOrSupervisor` treats `admin` and `supervisor` alike for most gating.

Router guard

`lib/core/navigation/router.dart` redirect logic:

- Unauthenticated → `/login`.
- Authenticated + `mustChangePin` → `/setup-pin`.
- Cashiers are bounced back to `/dashboard` from any path starting with: `/inventory`, `/users`, `/settings/csv-import`, `/settings/backup`, `/settings/store-info`.

> **NOTE** Recovery from total admin lockout: restore a backup, or reinstall (the seeder recreates the default Admin / `000000`). There is no back-door reset.

7. Navigation & routes

Route	Screen	Notes
`/login`	LoginScreen	initial location
`/setup-pin`	SetupPinScreen	forced PIN change
`/dashboard`	DashboardScreen	owns the live-orders socket
`/order`	OrderingScreen	
`/order/discount`	DiscountScreen	
`/order/payment`	PaymentScreen	
`/order/receipt/:id`	ReceiptScreen	type = order
`/transactions`	TransactionsScreen	
`/transactions/:id`	ReceiptScreen	type = transaction
`/orders`	OrdersScreen	live online orders
`/cashier-accounting`	CashierAccountingHubScreen	
`/cashier-accounting/x-reading[/history[/:id]]`	X-Reading + history + reprint	
`/cashier-accounting/daily-report[/history[/:id]]`	Daily Report + history + reprint	
`/cashier-accounting/z-reading[/history[/:id]]`	Z-Reading + history + reprint	
`/inventory`	InventoryScreen	tabs: Products / Categories / Modifiers
`/inventory/products/:id/modifiers`	ModifierGroupsScreen	
`/users`	UsersScreen	Admin / Supervisor
`/settings`	SettingsScreen	
`/settings/csv-import`	CsvImportScreen	Admin
`/settings/backup`	BackupScreen	Admin
`/settings/store-info`	StoreInfoScreen	Admin
`/settings/printer`	PrinterSetupScreen	all
`/settings/csv-export-key`	CsvExportKeyScreen	informational only

> **NOTE** The `reports` feature (`ReportsScreen`, fl_chart sales health) is fully built but the dashboard tile is commented out (`_kTileReports hidden for now`) and no route is registered. Wire a `GoRoute` + tile to expose it.

---

8. State management & data flow

Flow: **View → Riverpod notifier (state/) → Repository → DAO (local) or API source (`lib/data/backend_api/`) → Dio**.

- Screens are `HookConsumerWidget` ; providers are watched with `ref.watch` / read with `ref.read` .
- After a sale, void, or refund, the acting code explicitly `ref.invalidate` s `transactionsProvider` , `xReadingProvider` , `zReadingProvider` , `dailyReportProvider` — Drift raw writes are invisible to Riverpod otherwise.
- `restoreBackup` similarly invalidates `storeInfoProvider` , `paymentMethodsProvider` , `usersProvider` , `inventoryNotifierProvider` , `allModifierGroupsProvider` .
- The dashboard's first-run flow is a gated chain (store → employees → products); only one prompt shows at a time.

9. Live orders architecture

The only online feature. Everything else runs offline.

Pieces

Piece	Detail
Socket	`OrdersFeedNotifier` connects a WebSocket to `ORDERS_LIVE_FEED_WS_URL`, kept alive with `ref.keepAlive`. **Owned by** `DashboardScreen` — it boots / checks the connection; the Orders screen only observes it. `main.dart` listens at the root so toasts surface on any screen.
Bearer token	A device token from `/devices/token` authenticates the socket. A rejected token → `deviceTokenStatusProvider` → error toast.
Events API	Order-status changes (Pending/Preparing/Ready/Fulfilled/Cancelled) are POSTed to `ORDERS_EVENTS_API_BASE_URL` with `CLIENT_ID` + a token from the `/auth/token` exchange (`WEBHOOK_SECRET`). A rejected exchange → `webhookAuthStatusProvider`.
Persistence	`order_events` — one row per order, keyed by `orderId`; later events overwrite. `persistedOrdersProvider` feeds the Orders list; `ordersCountProvider` feeds the dashboard badge.
Notifications	`OrderNotificationsService` — Android channel `live_orders`, one local notification per event, plus an in-app `SnackBar`. Both best-effort; failure never affects the feed.

Device registration

Saving **Store Information** triggers `merchantDeviceNotifierProvider` to register this device (store ID + device info from `device_info_plus` / `package_info_plus`) with the merchant service. Outcomes:

- **pending** — waiting for the provider to approve the device; shown by the status card on the Store Info screen and a one-time dialog.
- **approved** — the socket can connect.
- **error** — a dialog; it retries the next time Store Info is saved. **Check status** on the card re-polls.

Auth-failure meanings (Orders screen)

Reason	Blocks the list?
network / serverError / rateLimited / unexpectedResponse / unknown	No — cached list stays, toast only
invalidWebhookSecret / invalidClient / unauthorized / invalidRequest	Yes — "Can't load orders" (build config or store ID is wrong)

10. Printing

- **Receipt layout** — built with `esc_pos_utils_plus` (ESC/POS). `PrintService` is the entry point: `printXReading`, `printZReading`, `printDailyReport`, receipt printing, `printCalibration` / `printWidthProbe`.
- **Bluetooth** — `print_bluetooth_thermal` for connect / print; discovery / pairing via `BluetoothDiscoveryService` (`android_intent_plus`, `permission_handler`). Each print job opens its own connection; the setup screen disconnects after verifying so later connects don't fail on a held socket.
- **Built-in printer** — `BuiltInPrinter` platform channel; `isAvailable()` decides whether the "Print (Built-in)" button shows.
- **Saved printer** — MAC + name in preferences via `PrintService.getSavedMac()` / `getSavedName()`.

> **NOTE** Permission ordering matters: on API 31+ `print_bluetooth_thermal` hangs forever (never resolves) if `BLUETOOTH_CONNECT` isn't granted before the first call — the setup screen requests permission first, then checks adapter state.

11. Backups

- **Format** — a `.zip` containing a full dump of every Drift table plus the `product_images/` directory. `BackupService.createBackup` always writes one; `createBackupIfChanged` hashes the dump and skips if unchanged.
- **Location** — `media_store_plus` saves into the public **Downloads/POS Backups** folder (`MediaStore.appFolder = 'POS Backups'`), file `pos_backup_<timestamp>.zip` . A manifest (`BackupManifestStore`) tracks entries + hashes.
- **Schedule** — `schedulePeriodicBackup()` registers a `workmanager` periodic task (`periodic_pos_backup` , ~3h, `ExistingPeriodicWorkPolicy.keep`). Dispatcher: `backupCallbackDispatcher` (`@pragma('vm:entry-point')`).
- **App-open safety net** — `_runStartupBackupSafetyNet` in `main.dart` runs a foreground backup if the last one is > 3h old (covers Doze / device-off).
- **Retention** — `retentionWindow = 7 days` ; older zips are deleted from Downloads and the manifest on each write.
- **Restore** — `BackupService.restoreBackup(db, zipFile:)` : makes a safety backup of current data, then raw-writes the archive over SQLite (full replace). Callers must invalidate cached providers (see State section). `BackupArchiveException` for a bad archive.

12. CSV import & export

Importers

`lib/core/csv/`. All expect a header row; headers are lower-cased and trimmed.

Importer	Columns
Products (`ProductsCsvImporter`)	`Category, Category Description, Product Name, Product Description, Product Base Price, Variant Name, Variant Price, [Product Image URL]`. Modes: `upsert` (default) or `replace` — replace deletes anything absent from the file (items with sales are disabled, not deleted). Own dialog with a copyable template header + confirmation.
Modifiers (`ModifiersCsvImporter`)	`Modifier Group, Modifier Name`
Users (`UsersCsvImporter`)	`name, role` (admin/cashier), `pin` (6-digit). Duplicate names are skipped.
Store Info (`StoreInfoCsvImporter`)	`key, value` — keys: `store_name, address, tax_rate, currency, receipt_footer`

Generic importers return an `ImportResult` (`successCount`, `skippedCount`, per-row `errors`) rendered as an in-card banner + expandable error list.

Report export

- `reportCsvExporterProvider` — `exportTransactions(from, to)` writes to Downloads; `writeTransactionsTempFile` feeds the email path.
- `ReportEmailSender` (`mailer`) sends via `SENDER_EMAIL` / `SENDER_APP_PASSWORD`. Recipients are persisted (`ReportEmailRecipients`). `ReportEmailException` for SMTP failures.
- `CSV_EXPORT_PASSWORD`, when set, gates every export (prompt before download / email). The `CsvExportKeyScreen` is purely informational — the value is only editable in `.env`.

13. Android configuration

Setting	Value / source
applicationId	`com.dpo.mobile`
minSdk / targetSdk / compileSdk	`flutter.*` (Flutter defaults); launcher-icon `min_sdk_android: 21`
label	`POS Mobile`
launchMode	`singleTop` , `windowSoftInputMode=adjustResize`
launcher icons	`flutter_launcher_icons` from `assets/icon/app_icon.png`
`requestLegacyExternalStorage`	`true` (backup file writes on older Android)

Declared permissions (main manifest)

- `BLUETOOTH` , `BLUETOOTH_ADMIN` , `ACCESS_FINE_LOCATION` — all `maxSdkVersion="30"` (legacy pre-Android 12 pairing)
- `BLUETOOTH_SCAN` (`neverForLocation`), `BLUETOOTH_CONNECT` — API 31+
- `POST_NOTIFICATIONS` — live-order notifications (Android 13+ also needs runtime consent, requested at startup)

> **IMPORTANT INTERNET IS ONLY DECLARED IN THE DEBUG AND PROFILE MANIFESTS**
> (`android/app/src/{debug,profile}/AndroidManifest.xml`), not in `src/main/AndroidManifest.xml`. Flutter injects it for debug builds, so live orders work in debug but a **RELEASE** build may have no network access. If live orders / device registration fail only in release, add `<uses-permission android:name="android.permission.INTERNET"/>` to the main manifest.

14. Building a release

```
cd mobile
# 1. ensure .env is the PRODUCTION file (secrets, prod URLs)
flutter pub get
dart run build_runner build --delete-conflicting-outputs

# 2a. APK (sideload / MDM push)
flutter build apk --release
#    -> build/app/outputs/flutter-apk/app-release.apk

# 2b. App Bundle (Play / managed Play)
flutter build appbundle --release
```

- Version comes from `pubspec.yaml` (`version: 1.2.0+3` → versionName 1.2.0, versionCode 3). Bump before each release.
- Configure release signing in `android/app/build.gradle(.kts)` + a keystore / `key.properties` (not committed).
- Verify the INTERNET permission note above before shipping a networked build.
- The `.env` used at build time is **baked into the binary** — build prod artifacts from a machine with the prod `.env`, and never commit it.

Windows

`flutter build windows` compiles, but the app is Android-first: `media_store_plus` (backups to Downloads), `print_bluetooth_thermal`, `workmanager` and `flutter_local_notifications` are Android-oriented. Treat Windows as unsupported for production unless each of those is validated.

15. Troubleshooting

Symptom	Cause / fix
`build_runner` fails / stale generated code	`dart run build_runner build --delete-conflicting-outputs`; if still stuck, `flutter clean && flutter pub get` first.
Codegen error mentioning `.env`	`.env` is missing or missing a key that `Env` declares. Copy from `.env.sample`.
Bluetooth scan hangs forever (no error)	`BLUETOOTH_CONNECT` not granted before the first plugin call. Grant it; the setup screen already requests permission first — check it wasn't permanently denied.
Live orders never connect in a release build	Missing INTERNET permission in the main manifest (see Android Configuration).
Orders screen: "Can't load orders"	`invalidWebhookSecret` / `invalidClient` / `unauthorized` / `invalidRequest` — wrong `.env` values or a store ID the service rejects. Check the device-registration card in Store Info.
Backups not appearing in Downloads	`media_store_plus` needs storage access on the Android version in use; check `MediaStore.ensureInitialized()` ran (it's in `main()`). WorkManager jobs can be delayed by Doze — the app-open safety net covers this.
Report emails not sending	`SENDER_APP_PASSWORD` must be a Gmail **App Password** with 2-Step Verification on `SENDER_EMAIL`. `ReportEmailException` carries the SMTP message.
Totals wrong after a manual DB edit / restore	Riverpod caches. Invalidate the relevant providers (see State section) or restart the app.
Admin locked out	Restore a backup, or reinstall (seeder recreates Admin / `000000`).

16. Notes & constraints

- **Single-tenant, single-device.** Each install is its own store with its own database. No cross-device sync — data moves only via backup / restore.
- **No user session persistence** by design — every launch requires sign-in.
- **Currency** is effectively fixed to `PHP` (`store_info.currency` default); VAT / Senior-PWD logic is Philippines-shaped (12% VAT, statutory Senior/PWD discount & VAT exemption).
- **Refunds**: schema (`refunds` , `refund_items`) and a `RefundScreen` exist, but the receipt UI currently exposes **Void** only.
- **Reports feature** is dormant — build it into the router / dashboard to use it.
- **Secrets in the binary**: obfuscated `envied` fields are XOR-masked, not secure. Anyone with the APK can recover them. Scope the webhook secret / Gmail app password accordingly and rotate on staff turnover.
- Keep the `path_provider_android` override until a newer version is verified.